CETAM

Centro de Educação Tecnológica do Amazonas

CADERNO DE PROJETOS INTEGRADORES

Linguagem de Programação

Python

5 projetos para praticar e consolidar o aprendizado

com Visual Studio Code e Google Colaboratory

Enunciados para o aluno + Gabarito Oficial do professor (ao final)

Elaboração: Profa. Priscila Gonçalves

Manaus – Amazonas • 2026

Sumário

Apresentação 4
Visão geral dos projetos 4
Como desenvolver 4
Regras gerais de entrega 5
Modelo do quadro de testes 5
Projeto 1 – Caixa da Feira do Produtor 6
Cenário 6
Objetivos de aprendizagem 6
Dados de partida 6
Requisitos funcionais 7
Regras de negócio 7
Exemplo de execução 7
O que entregar 8
Critérios de avaliação 8
Dicas 8
Desafios extras (opcionais) 9
Projeto 2 – Monitor Fluvial: Nível do Rio Negro 10
Cenário 10
Objetivos de aprendizagem 10
Requisitos funcionais 11
Regras de negócio 11
Exemplo de execução 11
O que entregar 11
Critérios de avaliação 12
Dicas 12
Desafios extras (opcionais) 12
Projeto 3 – Estoque da Cooperativa Ribeirinha 13
Cenário 13
Objetivos de aprendizagem 13
Dados de partida 13
Requisitos funcionais 14
Regras de negócio 14
Exemplo de execução 14

O que entregar 15
Critérios de avaliação 15
Dicas 16
Desafios extras (opcionais) 16
Projeto 4 – Boletim Escolar Digital 17
Cenário 17
Objetivos de aprendizagem 17
Requisitos funcionais 18
Regras de negócio 18
Exemplo de execução 18
O que entregar 19
Critérios de avaliação 19
Dicas 19
Desafios extras (opcionais) 19
Projeto 5 – Amazônia Bank: Mini-Banco Digital 21
Cenário 21
Objetivos de aprendizagem 21
Requisitos funcionais 22
Regras de negócio 22
Exemplo de execução 22
O que entregar 23
Critérios de avaliação 23
Dicas 23
Desafios extras (opcionais) 24
Gabarito Oficial dos Projetos Integradores 25
Como corrigir 25
Gabarito – Projeto 1: Caixa da Feira do Produtor 26
Solução de referência 26
Execução de teste 27
Casos de teste para correção 29
Matriz de correção 30
Erros frequentes e orientações 30
Orientações para os desafios extras 30
Gabarito – Projeto 2: Monitor Fluvial: Nível do Rio Negro 31
Solução de referência 31

Execução de teste 32
Casos de teste para correção 33
Matriz de correção 34
Erros frequentes e orientações 34
Orientações para os desafios extras 34
Gabarito – Projeto 3: Estoque da Cooperativa Ribeirinha 35
Solução de referência 35
Execução de teste 37
Casos de teste para correção 40
Matriz de correção 40
Erros frequentes e orientações 41
Orientações para os desafios extras 41
Gabarito – Projeto 4: Boletim Escolar Digital 42
Solução de referência 42
Execução de teste 44
Casos de teste para correção 48
Matriz de correção 48
Erros frequentes e orientações 48
Orientações para os desafios extras 49
Gabarito – Projeto 5: Amazônia Bank: Mini-Banco Digital 50
Solução de referência 50
Execução de teste 53
Casos de teste para correção 57
Matriz de correção 58
Erros frequentes e orientações 58
Orientações para os desafios extras 58

Apresentação

Os cinco projetos deste caderno foram pensados para você **praticar tudo o que aprendeu em Python** em situações parecidas com as da vida profissional. Eles têm **dificuldade crescente**: começam com um programa simples de caixa e terminam com um sistema bancário orientado a objetos com dados salvos em arquivo.

Todos os cenários usam o **contexto amazônico** e **dados fictícios**. Você pode desenvolver cada projeto no **Google Colab**, no **VS Code** ou nos dois.

Visão geral dos projetos

Nº	Projeto	Nível	Conteúdos mobilizados	Tempo sugerido
1	**Caixa da Feira do Produtor**	★★★★★	variáveis, tipos, `input()`, operadores, `if/elif/else`, `while`, `break`, `continue`	2 a 4 horas (individual ou dupla)
2	**Monitor Fluvial: Nível do Rio Negro**	★★★★★	listas, dicionários, strings, `for`/`range`, decisões, métodos de listas e funções internas	2 a 4 horas (individual ou dupla)
3	**Estoque da Cooperativa Ribeirinha**	★★★★★	dicionários aninhados, funções com parâmetros e retorno, menu com `while`, validações, strings	2 a 4 horas (individual ou dupla)
4	**Boletim Escolar Digital**	★★★★★	listas de dicionários, funções, strings, arquivos `.txt` (leitura e escrita), laços e decisões	2 a 4 horas (individual ou dupla)
5	**Amazônia Bank: Mini-Banco Digital**	★★★★★	classes e objetos, métodos, atributos, dicionário de objetos, arquivos, funções, menus aninhados e validações	2 a 4 horas (individual ou dupla)

Como desenvolver

1. **Leia o enunciado inteiro** antes de programar: cenário, requisitos (RF) e regras de negócio (RN).
2. **Planeje** em pseudocódigo ou em uma lista de passos. Decida quais variáveis, listas ou funções vai precisar.
3. **Construa em etapas:** faça um requisito funcionar, teste e só depois passe ao seguinte.
4. **Teste sempre** com valores normais, valores-limite e entradas inválidas.

5. **Revise o código:** nomes claros, comentários, indentação e ausência de repetição desnecessária.
6. **Preencha o quadro de testes** e entregue.

Regras gerais de entrega

- Nos projetos em dupla, **os dois integrantes** devem conseguir explicar qualquer parte do código.
- Coloque os **nomes dos integrantes** em comentário no início do arquivo.
- O programa deve funcionar **do início ao fim** sem travar com entradas inválidas previstas nas regras.
- É proibido copiar o código de outra dupla; consultar o material do curso e a documentação do Python é permitido.
- A atribuição de notas é de competência do professor.

Modelo do quadro de testes

Nº	O que foi testado (entradas)	Resultado esperado	Resultado obtido
1			
2			
3			

> 💻 **Colab × VS Code**
> No **Colab**, o campo do `input()` aparece abaixo da célula; arquivos `.txt` ficam no ambiente temporário e desaparecem ao fim da sessão (monte o Google Drive para guardá-los).
> No **VS Code**, a digitação é feita no terminal integrado; abra a **pasta** do projeto para que os arquivos sejam criados no lugar certo.

Projeto 1 – Caixa da Feira do Produtor

Ficha do projeto	Informações
Tema	Ponto de venda de uma feira regional
Nível	★★★★★
Conteúdos	Aulas 1 a 10: variáveis, tipos, `input()`, operadores, `if/elif/else`, `while`, `break`, `continue`
Tempo sugerido	2 a 4 horas
Organização	individual ou dupla

Cenário

Na feira do produtor de um bairro de Manaus, o caixa ainda é feito na calculadora e na caderneta. Os feirantes querem um **programa de caixa digital** que registre os produtos comprados por cada cliente, aplique descontos por volume, receba o pagamento e calcule o troco, e que ao fim do dia informe quanto foi vendido.

Os preços são **fictícios** e servem apenas para o exercício.

Objetivos de aprendizagem

- Aplicar entrada e saída de dados com conversão de tipos.
- Tomar decisões com `if/elif/else` e condições compostas.
- Usar laços `while` para repetir atendimentos e validar entradas.
- Controlar o fluxo com `break` e `continue`.

Dados de partida

Código	Produto	Preço por kg (fictício)
1	Tambaqui	R$ 28,00
2	Pirarucu	R$ 45,00
3	Tucunaré	R$ 32,00
4	Farinha de mandioca	R$ 9,00

Requisitos funcionais

Código	O sistema deve...
RF01	Exibir um **menu de produtos** (códigos 1 a 4) e a opção **0 – Finalizar compra**, repetindo o menu até o cliente finalizar.
RF02	Para cada produto escolhido, pedir a **quantidade em kg** e mostrar a linha: produto, quantidade, preço por kg e valor.
RF03	Acumular o **subtotal** da compra e a **quantidade de itens** registrados.
RF04	Aplicar **desconto por volume** sobre o subtotal (regras RN02 e RN03).
RF05	Mostrar o **resumo da compra**: itens, subtotal, desconto (com a porcentagem) e total a pagar.
RF06	Perguntar a **forma de pagamento** (1 – Dinheiro, 2 – Cartão). No dinheiro, ler o valor recebido e **calcular o troco**.
RF07	Perguntar **"Atender novo cliente? (s/n)"** e repetir o atendimento completo enquanto a resposta for "s".
RF08	Ao encerrar, mostrar o **fechamento do caixa**: clientes atendidos e total vendido no dia.

Regras de negócio

Código	Regra
RN01	A quantidade deve ser **maior que zero**; caso contrário, avisar e perguntar de novo.
RN02	Subtotal **a partir de R$ 100,00** e menor que R$ 200,00: **5%** de desconto.
RN03	Subtotal **a partir de R$ 200,00**: **10%** de desconto. Abaixo de R$ 100,00: sem desconto.
RN04	Opção de menu inexistente: avisar "Opção inválida!" e mostrar o menu novamente.
RN05	Não é possível finalizar uma compra **sem nenhum item** registrado.
RN06	No pagamento em dinheiro, o valor recebido não pode ser **menor que o total**: informar quanto falta e pedir de novo.
RN07	A forma de pagamento só aceita 1 ou 2; repetir a pergunta se for outro valor.

Exemplo de execução

Trecho de uma execução (os valores digitados aparecem após os dois pontos):

```
=== FEIRA DO PRODUTOR - CAIXA DIGITAL ===
(preços fictícios, apenas para o exercício)

--- Novo atendimento ---

1 - Tambaqui (R$ 28,00/kg)
2 - Pirarucu (R$ 45,00/kg)
3 - Tucunaré (R$ 32,00/kg)
```

```
4 - Farinha de mandioca (R$ 9,00/kg)
0 - Finalizar compra
Produto: 1
Quantidade de Tambaqui (kg): 5
Tambaqui: 5.00 kg x R$ 28.00 = R$ 140.00

1 - Tambaqui (R$ 28,00/kg)
2 - Pirarucu (R$ 45,00/kg)
3 - Tucunaré (R$ 32,00/kg)
4 - Farinha de mandioca (R$ 9,00/kg)
0 - Finalizar compra
Produto: 4
Quantidade de Farinha de mandioca (kg): 2
Farinha de mandioca: 2.00 kg x R$ 9.00 = R$ 18.00

1 - Tambaqui (R$ 28,00/kg)
2 - Pirarucu (R$ 45,00/kg)
```

O que entregar

- Arquivo `projeto1_caixa.py` (VS Code) **ou** notebook `Projeto1_Caixa.ipynb` (Colab), com os nomes dos integrantes em comentário no topo.
- Quadro de **testes** preenchido (veja o modelo na Apresentação) com, no mínimo, 5 casos.

Critérios de avaliação

Critério	O que será observado
Funcionamento	O caixa roda do início ao fechamento sem travar e com resultados corretos.
Regras de negócio	Descontos, troco e validações (RN01 a RN07) corretos.
Uso dos conteúdos	`input()` com conversão, `if/elif/else`, `while`, `break` e `continue`.
Organização	Nomes claros, indentação, comentários e saída legível.
Testes	Quadro de testes coerente com o comportamento do programa.

Dicas

- Use **dois laços `while`**: um interno (itens da compra) e outro externo (clientes).
- Zere `subtotal` e `itens` **no início de cada atendimento**.
- Use `continue` para voltar ao menu quando a opção for inválida.
- Para mostrar a porcentagem, use `taxa * 100` e `:.0f`.

Desafios extras (opcionais)

- **Nota detalhada:** guarde os itens da compra em uma lista e imprima a nota com todos eles.
- **Cupom:** peça um código; “FEIRA10” dá mais 10% sobre o total.
- **Fechamento em arquivo:** grave o resumo do dia em `fechamento.txt`.

Projeto 2 – Monitor Fluvial: Nível do Rio Negro

Ficha do projeto	Informações
Tema	Análise de medições do nível de um rio
Nível	★★★★★
Conteúdos	Aulas 1 a 14: listas, dicionários, strings, `for`/`range`, decisões, métodos de listas e funções internas
Tempo sugerido	2 a 4 horas
Organização	individual ou dupla

Cenário

Pesquisadores e comunidades ribeirinhas acompanham o **nível do Rio Negro** em Manaus mês a mês, pois ele determina a navegação, a pesca e a vida às margens. Você vai criar um **monitor fluvial**: o programa recebe as 12 medições do ano, analisa os dados e permite consultar qualquer mês.

Use **dados fictícios** na hora de testar (sugestão: 26,1 / 27,4 / 28,3 / 29,0 / 29,6 / 29,8 / 29,2 / 27,9 / 25,7 / 23,8 / 23,1 / 24,0 metros).

Objetivos de aprendizagem

- Armazenar e processar dados em **listas** paralelas.
- Percorrer listas com `for` e `range(len(...))`.
- Usar **dicionários** como contadores.
- Manipular **strings** para padronizar buscas.
- Usar `sum`, `max`, `min`, `len` e `index`.

Requisitos funcionais

Código	O sistema deve...
RF01	Criar a lista `meses` (Jan a Dez) e ler, com `for`, o **nível em metros** de cada mês, guardando-os na lista `niveis`.
RF02	Calcular e mostrar a **média anual**, o **maior** e o **menor** nível, informando **em qual mês** ocorreram.
RF03	Classificar cada mês (**Cheia**, **Enchente/Vazante** ou **Seca**) e mostrar um **relatório** com mês, nível, classificação e uma **barra de caracteres** proporcional ao nível.
RF04	Usar um **dicionário** para contar quantos meses há em cada classificação.
RF05	Descobrir o **mês de maior subida** do nível em relação ao mês anterior e quanto subiu.
RF06	**Consulta por mês**: o usuário digita o nome do mês (com qualquer combinação de maiúsculas, minúsculas e espaços) e o programa mostra nível e classificação; “fim” encerra.

Regras de negócio

Código	Regra
RN01	Níveis válidos: de **0 a 40 metros**. Valor fora da faixa: avisar e pedir de novo.
RN02	Nível **a partir de 29 m**: “Cheia”. **De 26 m até menos de 29 m**: “Enchente/Vazante”. **Abaixo de 26 m**: “Seca”.
RN03	A barra do relatório tem **um caractere `#` para cada metro inteiro** (use `int(nivel)`).
RN04	Na consulta, mês inexistente: mostrar “Mês não encontrado.”
RN05	A maior subida considera apenas variações **positivas** (de um mês para o seguinte).

Exemplo de execução

Trecho de uma execução (os valores digitados aparecem após os dois pontos):

```
=== MONITOR FLUVIAL - RIO NEGRO ===
Nível em Jan (m): 45
Valor inválido! Digite entre 0 e 40 metros.
Nível em Jan (m): 26.1
Nível em Fev (m): 27.4
Nível em Mar (m): 28.3
Nível em Abr (m): 29
Nível em Mai (m): 29.6
```

O que entregar

- Arquivo `projeto2_monitor.py` ou notebook `Projeto2_Monitor.ipynb`, com nomes dos integrantes no topo.

- Quadro de **testes** com, no mínimo, 5 casos (inclua um valor inválido e uma consulta com mês inexistente).

Critérios de avaliação

Critério	O que será observado
Funcionamento	Leitura, análises e consulta funcionando com os dados de teste.
Estruturas de dados	Listas paralelas, dicionário contador e uso correto de índices.
Strings	Padronização da busca com `strip()` e `title()`.
Regras de negócio	Faixas de classificação, validação e maior subida corretas.
Organização e testes	Código legível, comentado e quadro de testes coerente.

Dicas

- `meses[niveis.index(maior)]` informa o mês de um valor da lista `niveis`.
- Para a barra: `"#" * int(nivel)` repete o caractere.
- A maior subida usa `for i in range(1, len(niveis))` e compara `niveis[i]` com `niveis[i - 1]`.
- Padronize a consulta com `.strip().title()`: “ jun ” vira “Jun”.

Desafios extras (opcionais)

- **Gráfico:** gere um gráfico de linhas com `matplotlib` (no Colab já vem instalada; no VS Code, `pip install matplotlib`).
- **Meses críticos:** liste os meses com nível abaixo de 24 m.
- **Comparação de anos:** leia um segundo ano e diga qual teve a maior média.

Projeto 3 – Estoque da Cooperativa Ribeirinha

Ficha do projeto	Informações
Tema	Controle de estoque e vendas com funções
Nível	★★★★★
Conteúdos	Aulas 1 a 16: dicionários aninhados, funções com parâmetros e retorno, menu com `while`, validações, strings
Tempo sugerido	2 a 4 horas
Organização	individual ou dupla

Cenário

Uma **cooperativa de produtores ribeirinhos** vende açaí, castanha, tapioca e cupuaçu. O controle é feito em caderno e frequentemente falta produto ou há venda além do estoque. Sua missão é construir o **sistema de estoque**, organizado em **funções**, para cadastrar produtos, registrar vendas e reposições e emitir um relatório.

Todos os preços e quantidades são **fictícios**.

Objetivos de aprendizagem

- Modelar dados com **dicionários aninhados**.
- Dividir o programa em **funções** com parâmetros e retorno.
- Validar entradas e regras de negócio.
- Construir um menu completo e um relatório gerencial.

Dados de partida

Produto inicial	Preço unitário	Quantidade
Açaí	R$ 18,50	40
Castanha	R$ 32,00	15
Tapioca	R$ 12,00	8
Cupuaçu	R$ 15,00	25

Requisitos funcionais

Código	O sistema deve...
RF01	Iniciar o programa com o **estoque inicial** da tabela, em um dicionário `{nome: {"preco": ..., "qtd": ...}}`.
RF02	**Menu** com: 1 – Listar estoque, 2 – Cadastrar produto, 3 – Registrar venda, 4 – Repor estoque, 5 – Relatório, 0 – Sair.
RF03	**Listar** todos os produtos com preço e quantidade.
RF04	**Cadastrar** produto novo (nome, preço, quantidade inicial).
RF05	**Vender**: informar produto e quantidade, baixar o estoque e **devolver o valor da venda**, acumulando o total vendido na sessão.
RF06	**Repor**: somar uma quantidade ao estoque de um produto existente.
RF07	**Relatório**: número de produtos, valor total em estoque, total vendido, produto mais caro e lista de produtos com **estoque baixo**.
RF08	O programa deve ser organizado em **funções** (no mínimo: `exibir_menu`, `listar_estoque`, `cadastrar_produto`, `vender`, `repor`, `valor_total` e `relatorio`).

Regras de negócio

Código	Regra
RN01	Nomes de produtos são padronizados com `strip()` e `title()` (“AÇAÍ” e “ açaí” são o mesmo produto).
RN02	Não cadastrar produto **repetido** nem com nome vazio.
RN03	Preço deve ser **maior que zero**; quantidade inicial **não pode ser negativa**.
RN04	Não vender produto inexistente, quantidade menor ou igual a zero ou **quantidade maior que o estoque**.
RN05	Reposição exige produto existente e quantidade **maior que zero**.
RN06	**Estoque baixo**: menos de 10 unidades.

Exemplo de execução

Trecho de uma execução (os valores digitados aparecem após os dois pontos):

```
=== COOPERATIVA RIBEIRINHA - ESTOQUE ===
1 - Listar estoque
2 - Cadastrar produto
3 - Registrar venda
4 - Repor estoque
5 - Relatório
0 - Sair
```

```
Escolha: 1

--- ESTOQUE ATUAL ---
Açaí: R$ 18.50 | 40 un.
Castanha: R$ 32.00 | 15 un.
Tapioca: R$ 12.00 | 8 un.
Cupuaçu: R$ 15.00 | 25 un.

=== COOPERATIVA RIBEIRINHA - ESTOQUE ===
1 - Listar estoque
2 - Cadastrar produto
3 - Registrar venda
4 - Repor estoque
5 - Relatório
0 - Sair
Escolha: 3
Produto: açaí
Quantidade: 5
Venda registrada: 5 x Açaí = R$ 92.50

=== COOPERATIVA RIBEIRINHA - ESTOQUE ===
1 - Listar estoque
2 - Cadastrar produto
3 - Registrar venda
4 - Repor estoque
5 - Relatório
0 - Sair
Escolha: 3
```

O que entregar

- Arquivo `projeto3_estoque.py` ou notebook `Projeto3_Estoque.ipynb`, com nomes da dupla no topo.
- Quadro de **testes** com, no mínimo, 6 casos e **registro de colaboração** (quem fez cada função).

Critérios de avaliação

Critério	O que será observado
Funcionamento	Todas as opções do menu funcionam com os dados de teste.
Modularização	Funções com responsabilidade única, parâmetros e `return` adequados.
Estruturas de dados	Dicionário aninhado manipulado corretamente.
Regras de negócio	Validações RN01 a RN06 implementadas.
Organização e colaboração	Código legível, testes e registro da dupla.

Dicas

- Percorra o dicionário com `for nome, dados in estoque.items():`.
- `vender()` deve **retornar** o valor (e `0` quando a venda não ocorrer), para somar à variável `vendido`.
- Funções que apenas alteram o dicionário não precisam de `return`: o dicionário é modificado no lugar.
- Defina a constante `MINIMO = 10` no topo do programa.

Desafios extras (opcionais)

- **Preço promocional:** vendas de 10 ou mais unidades têm 5% de desconto.
- **Persistência:** salve e carregue o estoque em arquivo `.txt`.
- **Histórico:** guarde as vendas em uma lista e mostre-as no relatório.

Projeto 4 – Boletim Escolar Digital

Ficha do projeto	Informações
Tema	Cadastro de alunos e notas com persistência em arquivo
Nível	★★★★★
Conteúdos	Aulas 1 a 17: listas de dicionários, funções, strings, arquivos `.txt` (leitura e escrita), laços e decisões
Tempo sugerido	2 a 4 horas
Organização	individual ou dupla

Cenário

Uma escola quer abandonar o boletim em papel. Você foi contratado(a) para criar o **Boletim Escolar Digital**: o professor cadastra alunos e suas três notas, o sistema calcula a média e a situação, e **guarda tudo em arquivo**, para que os dados não se percam ao fechar o programa. Também deve gerar um **relatório** em arquivo de texto.

Objetivos de aprendizagem

- Combinar **listas e dicionários** (lista de alunos; notas em lista).
- Persistir dados com **leitura e escrita de arquivos .txt**.
- Organizar o sistema em **funções** reutilizáveis.
- Calcular estatísticas e gerar relatório.

Requisitos funcionais

Código	O sistema deve...
RF01	Ao iniciar, **carregar** os alunos de `boletim.txt` (se o arquivo ainda não existir, começar com a lista vazia).
RF02	**Menu**: 1 – Cadastrar aluno, 2 – Listar boletim, 3 – Buscar aluno, 4 – Estatísticas da turma, 5 – Exportar relatório, 0 – Sair.
RF03	**Cadastrar** aluno com nome e **3 notas** (0 a 10) e **salvar** no arquivo imediatamente.
RF04	**Listar** o boletim: nome, as três notas, **média** e **situação** de cada aluno.
RF05	**Buscar** um aluno pelo nome (sem diferenciar maiúsculas/minúsculas) e mostrar seu boletim.
RF06	**Estatísticas**: média da turma, aluno com a maior média e quantidade de alunos em cada situação.
RF07	**Exportar** um relatório para `relatorio.txt` com uma linha `nome;média;situação` por aluno.

Regras de negócio

Código	Regra
RN01	Notas válidas: de **0 a 10**. Fora da faixa, avisar e pedir de novo.
RN02	**Média** = soma das três notas dividida por 3. **≥ 7**: Aprovado; **≥ 5 e < 7**: Recuperação; **< 5**: Reprovado.
RN03	Não cadastrar aluno com **nome repetido**, vazio ou contendo `;` (o ponto e vírgula é o separador do arquivo).
RN04	Nomes são padronizados com `strip()` e `title()`.
RN05	Com a lista vazia, listar e estatísticas devem avisar “Nenhum aluno cadastrado.” sem gerar erro.
RN06	Os dados devem **permanecer** ao fechar e reabrir o programa.

Exemplo de execução

Trecho de uma execução (os valores digitados aparecem após os dois pontos):

```
=== BOLETIM ESCOLAR DIGITAL ===
1 - Cadastrar aluno
2 - Listar boletim
3 - Buscar aluno
4 - Estatísticas da turma
5 - Exportar relatório
0 - Sair
Escolha: 1
Nome do aluno: ana lima
```

```
Nota 1 (0 a 10): 8
Nota 2 (0 a 10): 7
Nota 3 (0 a 10): 9
Aluno cadastrado e salvo!

=== BOLETIM ESCOLAR DIGITAL ===
1 - Cadastrar aluno
2 - Listar boletim
3 - Buscar aluno
4 - Estatísticas da turma
5 - Exportar relatório
0 - Sair
```

O que entregar

- Arquivo `projeto4_boletim.py` (VS Code) **ou** notebook `Projeto4_Boletim.ipynb` (Colab, com o Drive montado para guardar os arquivos).
- Quadro de **testes** com, no mínimo, 6 casos, incluindo a **prova de persistência** (fechar e reabrir) e o registro de colaboração da dupla.

Critérios de avaliação

Critério	O que será observado
Funcionamento	Cadastro, listagem, busca, estatísticas e exportação funcionando.
Persistência	Dados gravados e recarregados corretamente; relatório gerado.
Funções e estruturas	Funções coesas; lista de dicionários e notas em lista.
Regras de negócio	Validações e situação corretas (RN01 a RN06).
Organização e colaboração	Código legível, testes e registro da dupla.

Dicas

- Cada linha do arquivo pode ser `nome;nota1;nota2;nota3`. Ao ler, use `linha.strip().split(";")`.
- Guarde as notas em uma **lista** dentro do dicionário: `{"nome": "Ana Lima", "notas": [8.0, 7.0, 9.0]}`.
- No Colab, os arquivos somem ao fim da sessão: monte o Google Drive (Aula 17). No VS Code, abra a **pasta** do projeto.
- Crie funções pequenas: `calcular_media`, `situacao`, `ler_nota`, `buscar_aluno`, `salvar_alunos`, `carregar_alunos`.

Desafios extras (opcionais)

- **Ranking:** mostre os alunos em ordem decrescente de média.

- **Pesos:** troque a média simples por média ponderada (pesos 2, 3 e 5).
- **Edição:** permita corrigir uma nota ou remover um aluno.

Projeto 5 – Amazônia Bank: Mini-Banco Digital

Ficha do projeto	Informações
Tema	Sistema bancário com POO, arquivos e login
Nível	★★★★★
Conteúdos	Aulas 1 a 19: classes e objetos, métodos, atributos, dicionário de objetos, arquivos, funções, menus aninhados e validações
Tempo sugerido	2 a 4 horas
Organização	individual ou dupla

Cenário

Uma cooperativa de crédito de Manaus quer lançar o **Amazônia Bank**, um mini-banco digital de terminal. Cada cliente cria uma conta com **senha de 4 dígitos**, faz login e realiza **depósitos, saques, transferências** e consulta **saldo e extrato**. Os dados das contas devem **permanecer salvos em arquivo**.

Este é o **projeto final do curso**: ele integra todos os conteúdos, incluindo a **Programação Orientada a Objetos**. Os valores são **fictícios**.

Objetivos de aprendizagem

- Modelar o domínio com **classes**, atributos e métodos.
- Manter um **dicionário de objetos** (`titular → Conta`).
- Persistir e recarregar os dados com **arquivos**.
- Construir **menus aninhados** com autenticação.
- Aplicar validações e regras de negócio bancárias.

Requisitos funcionais

Código	O sistema deve...
RF01	Criar a classe `Conta` com os atributos `titular`, `senha`, `saldo` (inicial 0) e `historico` (lista de movimentações).
RF02	Implementar os métodos `depositar(valor)`, `sacar(valor)` e `transferir(destino, valor)`, que devolvem `True` (operação realizada) ou `False`, e atualizam o `historico`.
RF03	Implementar `extrato()`, que mostra as movimentações da sessão e o saldo atual, e `autenticar(senha)`.
RF04	**Menu principal**: 1 – Criar conta, 2 – Entrar, 0 – Sair.
RF05	**Menu da conta** (após o login): 1 – Saldo, 2 – Depositar, 3 – Sacar, 4 – Transferir, 5 – Extrato, 0 – Sair da conta.
RF06	**Persistir** as contas em `contas.txt` (`titular;senha;saldo`) após cada criação e cada operação financeira, e **carregá-las** ao iniciar.
RF07	Organizar o código com a classe `Conta` e **funções** para carregar, salvar, criar conta e menus.

Regras de negócio

Código	Regra
RN01	Titular padronizado com `strip()` e `title()`; **não pode ser vazio, conter `;` nem repetir** um titular existente.
RN02	A senha deve ter **exatamente 4 dígitos numéricos**.
RN03	Login só é aceito com **titular existente e senha correta**; caso contrário, "Titular ou senha inválidos."
RN04	Depósito, saque e transferência exigem **valor maior que zero**.
RN05	Não é permitido **sacar ou transferir mais do que o saldo**.
RN06	A transferência exige **conta de destino existente** e **diferente da conta de origem**; registra movimentação nas duas contas.
RN07	O saldo deve ser exibido sempre com **duas casas decimais**.

Exemplo de execução

Trecho de uma execução (os valores digitados aparecem após os dois pontos):

```
=== AMAZÔNIA BANK - MINI-BANCO DIGITAL ===
1 - Criar conta
2 - Entrar
0 - Sair
Escolha: 1
Nome do titular: marina silva
```

```
Senha (4 dígitos): 1234
Conta criada com sucesso para Marina Silva!

=== AMAZÔNIA BANK - MINI-BANCO DIGITAL ===
1 - Criar conta
2 - Entrar
0 - Sair
Escolha: 1
Nome do titular: joão pedro
Senha (4 dígitos): 12ab
A senha deve ter exatamente 4 dígitos numéricos.

=== AMAZÔNIA BANK - MINI-BANCO DIGITAL ===
1 - Criar conta
2 - Entrar
0 - Sair
Escolha: 1
Nome do titular: JOÃO PEDRO
Senha (4 dígitos): 4321
Conta criada com sucesso para João Pedro!
```

O que entregar

- Arquivo `projeto5_amazonia_bank.py` (VS Code) **ou** notebook `Projeto5_AmazoniaBank.ipynb` (Colab, com o Drive montado).
- Quadro de **testes** com, no mínimo, 8 casos (incluindo persistência) e **registro de colaboração** da dupla.
- **Apresentação oral** de 5 a 7 minutos: demonstração, explicação da classe `Conta` e das decisões de projeto.

Critérios de avaliação

Critério	O que será observado
Funcionamento	Criação de conta, login e todas as operações financeiras funcionando.
POO	Classe `Conta` com atributos e métodos coerentes; objetos usados corretamente.
Persistência	Contas salvas e recarregadas; saldos corretos após reiniciar.
Regras de negócio	RN01 a RN07 implementadas.
Organização, testes e apresentação	Código legível, testes completos e boa comunicação oral.

Dicas

- Guarde as contas em um dicionário: `contas = {"Marina Silva": Conta(...)}`; assim, `titular in contas` é a verificação de existência.

- Faça `depositar`, `sacar` e `transferir` **devolverem `True` ou `False`**: o menu decide a mensagem e quando salvar.
- Na transferência, o objeto de destino é passado como argumento: `conta.transferir(contas[nome_destino], valor)`.
- Use `senha.isdigit()` e `len(senha) == 4` para validar a senha.
- Guardar senha em texto puro serve **apenas para estudo**: em sistemas reais, ela é protegida por *hash*.

Desafios extras (opcionais)

- **Histórico em arquivo:** persista as movimentações para que o extrato sobreviva ao fechamento.
- **Tentativas:** bloqueie o login após 3 senhas incorretas seguidas.
- **Tarifa:** cobre R$ 2,00 por saque (se houver saldo suficiente).
- **Classe `Banco`:** encapsule o dicionário de contas e as operações de carregar/salvar.

EXCLUSIVO DO PROFESSOR

Gabarito Oficial dos Projetos Integradores

Esta seção reúne, para cada projeto, a **solução de referência**, as **saídas obtidas na execução de teste**, os **casos de teste para correção**, a **matriz de correção**, os **erros mais frequentes** e **orientações para os desafios extras**. Todos os códigos foram **executados** com as entradas indicadas.

> 📌 **Para distribuir aos alunos**
> Esta seção está em uma **seção separada do documento**, com cabeçalho vermelho. Antes de entregar o caderno aos alunos, **exclua tudo a partir desta página** (ou salve uma cópia apenas com os enunciados).

Como corrigir

- Há **mais de uma solução correta**: compare o **comportamento** (saídas, validações e persistência), não a forma exata do código.
- Execute o programa do aluno com os **casos de teste** da tabela de cada projeto e com entradas inesperadas.
- Use a **matriz de correção** como guia dos níveis de desempenho. A atribuição de notas é de competência do professor.
- Confira se o aluno **consegue explicar** o próprio código: peça que altere uma regra (por exemplo, a faixa de desconto) na hora.

Gabarito – Projeto 1: Caixa da Feira do Produtor

Solução de referência

```
# Projeto 1 - Caixa da Feira do Produtor
print("=== FEIRA DO PRODUTOR - CAIXA DIGITAL ===")
print("(preços fictícios, apenas para o exercício)")

total_dia = 0
clientes = 0
novo_cliente = "s"

while novo_cliente == "s":
    subtotal = 0
    itens = 0
    print("\n--- Novo atendimento ---")

    while True:
        print("\n1 - Tambaqui (R$ 28,00/kg)")
        print("2 - Pirarucu (R$ 45,00/kg)")
        print("3 - Tucunaré (R$ 32,00/kg)")
        print("4 - Farinha de mandioca (R$ 9,00/kg)")
        print("0 - Finalizar compra")
        opcao = input("Produto: ")

        if opcao == "0":
            if itens == 0:
                print("Nenhum item registrado ainda.")
                continue
            break
        elif opcao == "1":
            nome = "Tambaqui"
            preco = 28.00
        elif opcao == "2":
            nome = "Pirarucu"
            preco = 45.00
        elif opcao == "3":
            nome = "Tucunaré"
            preco = 32.00
        elif opcao == "4":
            nome = "Farinha de mandioca"
            preco = 9.00
        else:
            print("Opção inválida!")
            continue

        quantidade = float(input(f"Quantidade de {nome} (kg): "))
        while quantidade <= 0:
            print("Quantidade inválida! Digite um valor maior que zero.")
            quantidade = float(input(f"Quantidade de {nome} (kg): "))

        valor = preco * quantidade
        subtotal += valor
```

```
        itens += 1
        print(f"{nome}: {quantidade:.2f} kg x R$ {preco:.2f} = R$ {valor:.2f}")

    if subtotal >= 200:
        taxa = 0.10
    elif subtotal >= 100:
        taxa = 0.05
    else:
        taxa = 0

    desconto = subtotal * taxa
    total = subtotal - desconto

    print("\n=== RESUMO DA COMPRA ===")
    print(f"Itens registrados: {itens}")
    print(f"Subtotal: R$ {subtotal:.2f}")
    print(f"Desconto ({taxa * 100:.0f}%): R$ {desconto:.2f}")
    print(f"Total a pagar: R$ {total:.2f}")

    forma = input("Pagamento (1 - Dinheiro, 2 - Cartão): ")
    while forma != "1" and forma != "2":
        print("Forma de pagamento inválida!")
        forma = input("Pagamento (1 - Dinheiro, 2 - Cartão): ")

    if forma == "1":
        recebido = float(input("Valor recebido (R$): "))
        while recebido < total:
            print(f"Valor insuficiente! Faltam R$ {total - recebido:.2f}.")
            recebido = float(input("Valor recebido (R$): "))
        print(f"Troco: R$ {recebido - total:.2f}")
    else:
        print("Pagamento no cartão aprovado.")

    total_dia += total
    clientes += 1
    novo_cliente = input("Atender novo cliente? (s/n): ").lower()

print("\n=== FECHAMENTO DO CAIXA ===")
print(f"Clientes atendidos: {clientes}")
print(f"Total vendido no dia: R$ {total_dia:.2f}")
```

Execução de teste

Entradas de teste – dois atendimentos (dinheiro com erro de valor; cartão com entradas inválidas):

```
=== FEIRA DO PRODUTOR - CAIXA DIGITAL ===
(preços fictícios, apenas para o exercício)

--- Novo atendimento ---

1 - Tambaqui (R$ 28,00/kg)
2 - Pirarucu (R$ 45,00/kg)
3 - Tucunaré (R$ 32,00/kg)
4 - Farinha de mandioca (R$ 9,00/kg)
```

```
0 - Finalizar compra
Produto: 1
Quantidade de Tambaqui (kg): 5
Tambaqui: 5.00 kg x R$ 28.00 = R$ 140.00

1 - Tambaqui (R$ 28,00/kg)
2 - Pirarucu (R$ 45,00/kg)
3 - Tucunaré (R$ 32,00/kg)
4 - Farinha de mandioca (R$ 9,00/kg)
0 - Finalizar compra
Produto: 4
Quantidade de Farinha de mandioca (kg): 2
Farinha de mandioca: 2.00 kg x R$ 9.00 = R$ 18.00

1 - Tambaqui (R$ 28,00/kg)
2 - Pirarucu (R$ 45,00/kg)
3 - Tucunaré (R$ 32,00/kg)
4 - Farinha de mandioca (R$ 9,00/kg)
0 - Finalizar compra
Produto: 0

=== RESUMO DA COMPRA ===
Itens registrados: 2
Subtotal: R$ 158.00
Desconto (5%): R$ 7.90
Total a pagar: R$ 150.10
Pagamento (1 - Dinheiro, 2 - Cartão): 1
Valor recebido (R$): 100
Valor insuficiente! Faltam R$ 50.10.
Valor recebido (R$): 200
Troco: R$ 49.90
Atender novo cliente? (s/n): s

--- Novo atendimento ---

1 - Tambaqui (R$ 28,00/kg)
2 - Pirarucu (R$ 45,00/kg)
3 - Tucunaré (R$ 32,00/kg)
4 - Farinha de mandioca (R$ 9,00/kg)
0 - Finalizar compra
Produto: 9
Opção inválida!

1 - Tambaqui (R$ 28,00/kg)
2 - Pirarucu (R$ 45,00/kg)
3 - Tucunaré (R$ 32,00/kg)
4 - Farinha de mandioca (R$ 9,00/kg)
0 - Finalizar compra
Produto: 2
Quantidade de Pirarucu (kg): -1
Quantidade inválida! Digite um valor maior que zero.
Quantidade de Pirarucu (kg): 1.5
Pirarucu: 1.50 kg x R$ 45.00 = R$ 67.50
```

```
1 - Tambaqui (R$ 28,00/kg)
2 - Pirarucu (R$ 45,00/kg)
3 - Tucunaré (R$ 32,00/kg)
4 - Farinha de mandioca (R$ 9,00/kg)
0 - Finalizar compra
Produto: 0

=== RESUMO DA COMPRA ===
Itens registrados: 1
Subtotal: R$ 67.50
Desconto (0%): R$ 0.00
Total a pagar: R$ 67.50
Pagamento (1 - Dinheiro, 2 - Cartão): 3
Forma de pagamento inválida!
Pagamento (1 - Dinheiro, 2 - Cartão): 2
Pagamento no cartão aprovado.
Atender novo cliente? (s/n): n

=== FECHAMENTO DO CAIXA ===
Clientes atendidos: 2
Total vendido no dia: R$ 217.60
```

Casos de teste para correção

Entrada / situação	Resultado esperado
5 kg de Tambaqui + 2 kg de Farinha; dinheiro: R$ 100 e depois R$ 200	Subtotal R$ 158,00; desconto 5% = R$ 7,90; total R$ 150,10; “Faltam R$ 50,10”; troco R$ 49,90
Opção 9 no menu	“Opção inválida!” e o menu é mostrado de novo
Quantidade −1 de Pirarucu, depois 1,5	“Quantidade inválida!” e nova pergunta; R$ 67,50
Finalizar (0) sem nenhum item	“Nenhum item registrado ainda.”
Forma de pagamento 3	“Forma de pagamento inválida!” e nova pergunta
10 kg de Pirarucu (subtotal R$ 450,00)	Desconto de 10% = R$ 45,00; total R$ 405,00
Fechamento após os dois atendimentos de teste	2 clientes; total do dia R$ 217,60

Matriz de correção

Critério	Atende plenamente	Atende parcialmente	Não atende
Funcionamento	Executa todo o fluxo sem erros; resultados corretos nos 7 casos de teste.	Executa, mas erra em 1 ou 2 casos (ex.: troco ou fechamento).	Trava ou não completa o atendimento.
Regras de negócio	RN01 a RN07 implementadas.	Faltam 1 ou 2 regras (ex.: RN05 ou RN06).	Desconto ou troco incorretos.
Uso dos conteúdos	Conversão de tipos, `if/elif/else`, `while`, `break`/`continue` adequados.	Usa laços, mas sem `break`/`continue` ou com repetições de código.	Sem repetição ou sem decisões.
Organização	Nomes claros, comentários, indentação e saída legível.	Nomes pouco claros ou sem comentários.	Código desorganizado.
Testes	Quadro com 5 ou mais casos coerentes.	Menos de 5 casos ou pouco detalhados.	Sem quadro de testes.

Erros frequentes e orientações

- **Não zerar `subtotal` e `itens` a cada cliente** (o segundo cliente herda os valores do primeiro).
- **Esquecer `float()`** em `quantidade` ou `recebido` (TypeError na comparação).
- **Usar `or` em vez de `and`** na validação da forma de pagamento (`forma != "1" or forma != "2"` é sempre verdadeiro).
- **Aplicar o desconto sobre o valor errado** (por exemplo, sobre o preço e não sobre o subtotal).
- Usar `if` em vez de `elif` nas faixas de desconto, aplicando a faixa errada.

Orientações para os desafios extras

- **Nota detalhada:** crie `itens_compra = []` e, a cada produto, `itens_compra.append(f"{nome}: {quantidade:.2f} kg = R$ {valor:.2f}")`; ao final, percorra a lista com `for`.
- **Cupom:** `cupom = input("Cupom: ").strip().upper()`; se `cupom == "FEIRA10"`, `total = total * 0.90`.
- **Fechamento em arquivo:** `with open("fechamento.txt", "w", encoding="utf-8") as f:` e `f.write(...)` com clientes e total do dia.

Gabarito – Projeto 2: Monitor Fluvial: Nível do Rio Negro

Solução de referência

```
# Projeto 2 - Monitor Fluvial: Nível do Rio Negro (dados fictícios)
meses = ["Jan", "Fev", "Mar", "Abr", "Mai", "Jun",
         "Jul", "Ago", "Set", "Out", "Nov", "Dez"]
niveis = []

print("=== MONITOR FLUVIAL - RIO NEGRO ===")
for mes in meses:
    nivel = float(input(f"Nível em {mes} (m): "))
    while nivel < 0 or nivel > 40:
        print("Valor inválido! Digite entre 0 e 40 metros.")
        nivel = float(input(f"Nível em {mes} (m): "))
    niveis.append(nivel)

# ----- Estatísticas -----
media = sum(niveis) / len(niveis)
maior = max(niveis)
menor = min(niveis)
mes_maior = meses[niveis.index(maior)]
mes_menor = meses[niveis.index(menor)]

print("\n--- ESTATÍSTICAS DO ANO ---")
print(f"Média anual: {media:.2f} m")
print(f"Maior nível: {maior:.1f} m em {mes_maior}")
print(f"Menor nível: {menor:.1f} m em {mes_menor}")

# ----- Relatório e contagem por classificação -----
contagem = {"Cheia": 0, "Enchente/Vazante": 0, "Seca": 0}
situacoes = []

print("\n--- RELATÓRIO MENSAL ---")
for i in range(len(meses)):
    nivel = niveis[i]
    if nivel >= 29:
        situacao = "Cheia"
    elif nivel >= 26:
        situacao = "Enchente/Vazante"
    else:
        situacao = "Seca"
    situacoes.append(situacao)
    contagem[situacao] += 1
    print(f"{meses[i]} | {nivel:.1f} m | {situacao} | {'#' * int(nivel)}")

print("\n--- MESES POR CLASSIFICAÇÃO ---")
for classe, qtd in contagem.items():
    print(f"{classe}: {qtd}")

# ----- Maior subida -----
```

```
maior_subida = 0
mes_subida = ""
for i in range(1, len(niveis)):
    variacao = niveis[i] - niveis[i - 1]
    if variacao > maior_subida:
        maior_subida = variacao
        mes_subida = meses[i]
print(f"\nMaior subida: +{maior_subida:.1f} m em {mes_subida}")

# ----- Consulta por mês -----
while True:
    busca = input("\nConsultar mês (ou 'fim'): ").strip().title()
    if busca == "Fim":
        break
    if busca in meses:
        i = meses.index(busca)
        print(f"{busca}: {niveis[i]:.1f} m - {situacoes[i]}")
    else:
        print("Mês não encontrado.")
print("Monitoramento encerrado.")
```

Execução de teste

Entradas de teste – com um valor inválido em janeiro e três consultas:

```
=== MONITOR FLUVIAL - RIO NEGRO ===
Nível em Jan (m): 45
Valor inválido! Digite entre 0 e 40 metros.
Nível em Jan (m): 26.1
Nível em Fev (m): 27.4
Nível em Mar (m): 28.3
Nível em Abr (m): 29
Nível em Mai (m): 29.6
Nível em Jun (m): 29.8
Nível em Jul (m): 29.2
Nível em Ago (m): 27.9
Nível em Set (m): 25.7
Nível em Out (m): 23.8
Nível em Nov (m): 23.1
Nível em Dez (m): 24

--- ESTATÍSTICAS DO ANO ---
Média anual: 26.99 m
Maior nível: 29.8 m em Jun
Menor nível: 23.1 m em Nov

--- RELATÓRIO MENSAL ---
Jan | 26.1 m | Enchente/Vazante | ##########################
Fev | 27.4 m | Enchente/Vazante | ############################
Mar | 28.3 m | Enchente/Vazante | ##############################
Abr | 29.0 m | Cheia | ##############################
Mai | 29.6 m | Cheia | ##############################
Jun | 29.8 m | Cheia | ##############################
Jul | 29.2 m | Cheia | ##############################
```

```
Ago | 27.9 m | Enchente/Vazante | ############################
Set | 25.7 m | Seca | ########################
Out | 23.8 m | Seca | ######################
Nov | 23.1 m | Seca | ######################
Dez | 24.0 m | Seca | ########################

--- MESES POR CLASSIFICAÇÃO ---
Cheia: 4
Enchente/Vazante: 4
Seca: 4

Maior subida: +1.3 m em Fev

Consultar mês (ou 'fim'):   jun
Jun: 29.8 m - Cheia

Consultar mês (ou 'fim'): xyz
Mês não encontrado.

Consultar mês (ou 'fim'): FIM
Monitoramento encerrado.
```

Casos de teste para correção

Entrada / situação	Resultado esperado
Janeiro = 45	“Valor inválido! Digite entre 0 e 40 metros.” e nova pergunta
Dados de teste sugeridos (26,1 … 24,0)	Média 26,99 m; maior 29,8 m em Jun; menor 23,1 m em Nov
Classificação dos 12 meses	Cheia: 4 (Abr–Jul); Enchente/Vazante: 4 (Jan–Mar e Ago); Seca: 4 (Set–Dez)
Maior subida	+1,3 m em Fev
Consulta “ jun ”	Jun: 29,8 m - Cheia
Consulta “xyz”	“Mês não encontrado.”
Consulta “FIM”	Encerra o programa
Nível exatamente 29,0 (abril)	Classificado como “Cheia” (limite inclusivo)

Matriz de correção

Critério	Atende plenamente	Atende parcialmente	Não atende
Funcionamento	Todas as etapas funcionam e os resultados conferem com os casos de teste.	Falha em uma etapa (ex.: consulta ou subida).	Não completa a leitura ou o relatório.
Estruturas de dados	Listas paralelas e dicionário contador corretos.	Usa listas, mas sem o dicionário ou com índices frágeis.	Variáveis soltas, sem listas.
Strings	Busca padronizada com `strip()` e `title()`.	Padroniza só parte dos casos.	Busca falha com maiúsculas/espaços.
Regras de negócio	Faixas, validação 0–40 e maior subida corretas.	Erro em limite (ex.: 29,0) ou na subida.	Classificação incorreta.
Organização e testes	Legível, comentado; quadro com 5+ casos.	Pouco comentado ou poucos testes.	Sem organização ou testes.

Erros frequentes e orientações

- **Inicializar `maior_subida` com o primeiro valor do nível** em vez de 0, ou comparar com `>=`, registrando subidas nulas.
- **Esquecer `float()`** na leitura, ou usar `int()` e rejeitar valores como 26,1.
- **Usar `range(12)` fixo** e quebrar se a lista mudar de tamanho: prefira `range(len(meses))`.
- **Não padronizar a consulta** (“jun” ≠ “Jun”).
- Confundir `index()` (posição) com o valor, ou chamar `niveis.index()` com o mês.

Orientações para os desafios extras

- **Gráfico:** `import matplotlib.pyplot as plt`; `plt.plot(meses, niveis, marker="o")`; `plt.show()`.
- **Meses críticos:** `for i in range(len(meses)):` com `if niveis[i] < 24:`.
- **Comparação de anos:** leia uma segunda lista `niveis2`, calcule as duas médias e compare com `if`.

Gabarito – Projeto 3: Estoque da Cooperativa Ribeirinha

Solução de referência

```
# Projeto 3 - Estoque da Cooperativa Ribeirinha (dados fictícios)
MINIMO = 10

estoque = {
    "Açaí": {"preco": 18.50, "qtd": 40},
    "Castanha": {"preco": 32.00, "qtd": 15},
    "Tapioca": {"preco": 12.00, "qtd": 8},
    "Cupuaçu": {"preco": 15.00, "qtd": 25},
}

def exibir_menu():
    print("\n=== COOPERATIVA RIBEIRINHA - ESTOQUE ===")
    print("1 - Listar estoque")
    print("2 - Cadastrar produto")
    print("3 - Registrar venda")
    print("4 - Repor estoque")
    print("5 - Relatório")
    print("0 - Sair")
    return input("Escolha: ")

def listar_estoque(estoque):
    print("\n--- ESTOQUE ATUAL ---")
    for nome, dados in estoque.items():
        print(f"{nome}: R$ {dados['preco']:.2f} | {dados['qtd']} un.")

def valor_total(estoque):
    total = 0
    for dados in estoque.values():
        total += dados["preco"] * dados["qtd"]
    return total

def produtos_baixos(estoque, minimo=MINIMO):
    baixos = []
    for nome, dados in estoque.items():
        if dados["qtd"] < minimo:
            baixos.append(nome)
    return baixos

def cadastrar_produto(estoque):
    nome = input("Nome do produto: ").strip().title()
    if nome == "":
        print("Nome inválido.")
        return
    if nome in estoque:
        print("Produto já cadastrado!")
        return
    preco = float(input("Preço (R$): "))
```

```
    while preco <= 0:
        print("O preço deve ser maior que zero.")
        preco = float(input("Preço (R$): "))
    qtd = int(input("Quantidade inicial: "))
    while qtd < 0:
        print("A quantidade não pode ser negativa.")
        qtd = int(input("Quantidade inicial: "))
    estoque[nome] = {"preco": preco, "qtd": qtd}
    print(f"Produto {nome} cadastrado!")

def vender(estoque):
    nome = input("Produto: ").strip().title()
    if nome not in estoque:
        print("Produto não encontrado.")
        return 0
    qtd = int(input("Quantidade: "))
    if qtd <= 0:
        print("Quantidade inválida.")
        return 0
    if qtd > estoque[nome]["qtd"]:
        print(f"Estoque insuficiente! Disponível: {estoque[nome]['qtd']} un.")
        return 0
    estoque[nome]["qtd"] -= qtd
    valor = qtd * estoque[nome]["preco"]
    print(f"Venda registrada: {qtd} x {nome} = R$ {valor:.2f}")
    return valor

def repor(estoque):
    nome = input("Produto: ").strip().title()
    if nome not in estoque:
        print("Produto não encontrado.")
        return
    qtd = int(input("Quantidade a repor: "))
    if qtd <= 0:
        print("Quantidade inválida.")
        return
    estoque[nome]["qtd"] += qtd
    print(f"Novo estoque de {nome}: {estoque[nome]['qtd']} un.")

def relatorio(estoque, vendido):
    print("\n=== RELATÓRIO DO ESTOQUE ===")
    print(f"Produtos cadastrados: {len(estoque)}")
    print(f"Valor total em estoque: R$ {valor_total(estoque):.2f}")
    print(f"Total vendido na sessão: R$ {vendido:.2f}")
    mais_caro = ""
    maior_preco = 0
    for nome, dados in estoque.items():
        if dados["preco"] > maior_preco:
            maior_preco = dados["preco"]
            mais_caro = nome
    print(f"Produto mais caro: {mais_caro} (R$ {maior_preco:.2f})")
    baixos = produtos_baixos(estoque)
    if len(baixos) == 0:
        print("Nenhum produto com estoque baixo.")
```

```
    else:
        print(f"Estoque baixo (menos de {MINIMO} un.):")
        for nome in baixos:
            print(f"- {nome}: {estoque[nome]['qtd']} un.")

vendido = 0
while True:
    opcao = exibir_menu()
    if opcao == "1":
        listar_estoque(estoque)
    elif opcao == "2":
        cadastrar_produto(estoque)
    elif opcao == "3":
        vendido += vender(estoque)
    elif opcao == "4":
        repor(estoque)
    elif opcao == "5":
        relatorio(estoque, vendido)
    elif opcao == "0":
        print("Até logo!")
        break
    else:
        print("Opção inválida!")
```

Execução de teste

Entradas de teste – sequência completa de operações:

```
=== COOPERATIVA RIBEIRINHA - ESTOQUE ===
1 - Listar estoque
2 - Cadastrar produto
3 - Registrar venda
4 - Repor estoque
5 - Relatório
0 - Sair
Escolha: 1

--- ESTOQUE ATUAL ---
Açaí: R$ 18.50 | 40 un.
Castanha: R$ 32.00 | 15 un.
Tapioca: R$ 12.00 | 8 un.
Cupuaçu: R$ 15.00 | 25 un.

=== COOPERATIVA RIBEIRINHA - ESTOQUE ===
1 - Listar estoque
2 - Cadastrar produto
3 - Registrar venda
4 - Repor estoque
5 - Relatório
0 - Sair
Escolha: 3
Produto: açaí
Quantidade: 5
Venda registrada: 5 x Açaí = R$ 92.50
```

```
=== COOPERATIVA RIBEIRINHA - ESTOQUE ===
1 - Listar estoque
2 - Cadastrar produto
3 - Registrar venda
4 - Repor estoque
5 - Relatório
0 - Sair
Escolha: 3
Produto: tapioca
Quantidade: 20
Estoque insuficiente! Disponível: 8 un.

=== COOPERATIVA RIBEIRINHA - ESTOQUE ===
1 - Listar estoque
2 - Cadastrar produto
3 - Registrar venda
4 - Repor estoque
5 - Relatório
0 - Sair
Escolha: 3
Produto: xingu
Produto não encontrado.

=== COOPERATIVA RIBEIRINHA - ESTOQUE ===
1 - Listar estoque
2 - Cadastrar produto
3 - Registrar venda
4 - Repor estoque
5 - Relatório
0 - Sair
Escolha: 5

=== RELATÓRIO DO ESTOQUE ===
Produtos cadastrados: 4
Valor total em estoque: R$ 1598.50
Total vendido na sessão: R$ 92.50
Produto mais caro: Castanha (R$ 32.00)
Estoque baixo (menos de 10 un.):
- Tapioca: 8 un.

=== COOPERATIVA RIBEIRINHA - ESTOQUE ===
1 - Listar estoque
2 - Cadastrar produto
3 - Registrar venda
4 - Repor estoque
5 - Relatório
0 - Sair
Escolha: 2
Nome do produto: guaraná
Preço (R$): 22.5
Quantidade inicial: 12
Produto Guaraná cadastrado!
```

```
=== COOPERATIVA RIBEIRINHA - ESTOQUE ===
1 - Listar estoque
2 - Cadastrar produto
3 - Registrar venda
4 - Repor estoque
5 - Relatório
0 - Sair
Escolha: 2
Nome do produto: AÇAÍ
Produto já cadastrado!

=== COOPERATIVA RIBEIRINHA - ESTOQUE ===
1 - Listar estoque
2 - Cadastrar produto
3 - Registrar venda
4 - Repor estoque
5 - Relatório
0 - Sair
Escolha: 4
Produto: tapioca
Quantidade a repor: 10
Novo estoque de Tapioca: 18 un.

=== COOPERATIVA RIBEIRINHA - ESTOQUE ===
1 - Listar estoque
2 - Cadastrar produto
3 - Registrar venda
4 - Repor estoque
5 - Relatório
0 - Sair
Escolha: 5

=== RELATÓRIO DO ESTOQUE ===
Produtos cadastrados: 5
Valor total em estoque: R$ 1988.50
Total vendido na sessão: R$ 92.50
Produto mais caro: Castanha (R$ 32.00)
Nenhum produto com estoque baixo.

=== COOPERATIVA RIBEIRINHA - ESTOQUE ===
1 - Listar estoque
2 - Cadastrar produto
3 - Registrar venda
4 - Repor estoque
5 - Relatório
0 - Sair
Escolha: 9
Opção inválida!

=== COOPERATIVA RIBEIRINHA - ESTOQUE ===
1 - Listar estoque
2 - Cadastrar produto
3 - Registrar venda
4 - Repor estoque
```

```
5 - Relatório
0 - Sair
Escolha: 0
Até logo!
```

Casos de teste para correção

Entrada / situação	Resultado esperado
Vender 5 un. de “açaí”	Venda: R$ 92,50; estoque de Açaí passa de 40 para 35
Vender 20 un. de Tapioca (estoque 8)	“Estoque insuficiente! Disponível: 8 un.” (nada é baixado)
Vender produto “Xingu”	“Produto não encontrado.”
Relatório após a venda	Valor em estoque R$ 1.598,50; vendido R$ 92,50; mais caro: Castanha; estoque baixo: Tapioca (8 un.)
Cadastrar “guaraná”, R$ 22,50, 12 un.	Cadastra como “Guaraná”
Cadastrar “AÇAÍ” (já existe)	“Produto já cadastrado!”
Repor 10 un. de Tapioca	Novo estoque: 18 un.; sai da lista de estoque baixo
Opção 9	“Opção inválida!”

Matriz de correção

Critério	Atende plenamente	Atende parcialmente	Não atende
Funcionamento	Todas as opções funcionam e conferem com os casos de teste.	Uma opção com falha (ex.: relatório ou reposição).	Menu não executa as operações.
Modularização	Funções com responsabilidade única, parâmetros e `return` corretos.	Funções criadas, mas com lógica repetida ou uso de variáveis globais.	Programa sem funções.
Estruturas de dados	Dicionário aninhado usado e atualizado corretamente.	Usa dicionário, mas com erros de acesso ou atualização.	Dados em variáveis soltas.
Regras de negócio	RN01 a RN06 implementadas.	Faltam 1 ou 2 validações.	Permite venda além do estoque ou duplica produtos.
Organização e colaboração	Legível, comentado, testes e registro da dupla completos.	Falta registro ou testes.	Sem organização ou colaboração.

Erros frequentes e orientações

- **Esquecer o `return` em `vender()`** e tentar somar `None` ao total (TypeError).
- **Usar `estoque[nome]` sem verificar `nome in estoque`** (KeyError).
- **Baixar o estoque antes de validar a quantidade**, deixando-o negativo.
- **Não padronizar o nome** ("açaí" ≠ "Açaí") e não encontrar produtos existentes.
- **Modificar a lista ou o dicionário dentro de um `for` sobre ele mesmo** em extensões do projeto.
- Colocar o `print` no lugar do `return` e perder o valor da venda.

Orientações para os desafios extras

- **Preço promocional:** em `vender()`, `if qtd >= 10: valor = valor * 0.95`.
- **Persistência:** gravar uma linha `nome;preco;qtd` por produto (modo `"w"`) e recarregar com `split(";")`, convertendo preço para `float` e quantidade para `int`.
- **Histórico:** `historico = []` e `historico.append(f"{qtd} x {nome} = R$ {valor:.2f}")` a cada venda.

Gabarito – Projeto 4: Boletim Escolar Digital

Solução de referência

```
# Projeto 4 - Boletim Escolar Digital
ARQUIVO = "boletim.txt"
RELATORIO = "relatorio.txt"

def carregar_alunos():
    alunos = []
    try:
        with open(ARQUIVO, "r", encoding="utf-8") as f:
            for linha in f:
                nome, n1, n2, n3 = linha.strip().split(";")
                alunos.append({"nome": nome, "notas": [float(n1), float(n2),
float(n3)]})
    except FileNotFoundError:
        pass
    return alunos

def salvar_alunos(alunos):
    with open(ARQUIVO, "w", encoding="utf-8") as f:
        for aluno in alunos:
            n = aluno["notas"]
            f.write(f"{aluno['nome']};{n[0]};{n[1]};{n[2]}\n")

def calcular_media(aluno):
    return sum(aluno["notas"]) / len(aluno["notas"])

def situacao(media):
    if media >= 7:
        return "Aprovado"
    elif media >= 5:
        return "Recuperação"
    return "Reprovado"

def ler_nota(numero):
    nota = float(input(f"Nota {numero} (0 a 10): "))
    while nota < 0 or nota > 10:
        print("Nota inválida! Digite um valor entre 0 e 10.")
        nota = float(input(f"Nota {numero} (0 a 10): "))
    return nota

def buscar_aluno(alunos, nome):
    for aluno in alunos:
        if aluno["nome"] == nome:
            return aluno
    return None

def mostrar_aluno(aluno):
    n = aluno["notas"]
    media = calcular_media(aluno)
```

```
    print(f"{aluno['nome']} | Notas: {n[0]:.1f} / {n[1]:.1f} / {n[2]:.1f} |
Média: {media:.2f} | {situacao(media)}")

def cadastrar_aluno(alunos):
    nome = input("Nome do aluno: ").strip().title()
    if nome == "" or ";" in nome:
        print("Nome inválido.")
        return
    if buscar_aluno(alunos, nome) is not None:
        print("Aluno já cadastrado!")
        return
    notas = [ler_nota(1), ler_nota(2), ler_nota(3)]
    alunos.append({"nome": nome, "notas": notas})
    salvar_alunos(alunos)
    print("Aluno cadastrado e salvo!")

def listar_boletim(alunos):
    if len(alunos) == 0:
        print("Nenhum aluno cadastrado.")
        return
    print("\n--- BOLETIM ---")
    for aluno in alunos:
        mostrar_aluno(aluno)

def estatisticas(alunos):
    if len(alunos) == 0:
        print("Nenhum aluno cadastrado.")
        return
    soma = 0
    melhor = alunos[0]
    contagem = {"Aprovado": 0, "Recuperação": 0, "Reprovado": 0}
    for aluno in alunos:
        media = calcular_media(aluno)
        soma += media
        if media > calcular_media(melhor):
            melhor = aluno
        contagem[situacao(media)] += 1
    print("\n--- ESTATÍSTICAS DA TURMA ---")
    print(f"Média da turma: {soma / len(alunos):.2f}")
    print(f"Maior média: {melhor['nome']} ({calcular_media(melhor):.2f})")
    for sit, qtd in contagem.items():
        print(f"{sit}: {qtd}")

def exportar_relatorio(alunos):
    with open(RELATORIO, "w", encoding="utf-8") as f:
        f.write("RELATÓRIO DA TURMA\n")
        for aluno in alunos:
            media = calcular_media(aluno)
            f.write(f"{aluno['nome']};{media:.2f};{situacao(media)}\n")
    print(f"Relatório gravado em {RELATORIO}.")

alunos = carregar_alunos()

while True:
```

```
    print("\n=== BOLETIM ESCOLAR DIGITAL ===")
    print("1 - Cadastrar aluno")
    print("2 - Listar boletim")
    print("3 - Buscar aluno")
    print("4 - Estatísticas da turma")
    print("5 - Exportar relatório")
    print("0 - Sair")
    opcao = input("Escolha: ")

    if opcao == "1":
        cadastrar_aluno(alunos)
    elif opcao == "2":
        listar_boletim(alunos)
    elif opcao == "3":
        nome = input("Nome: ").strip().title()
        aluno = buscar_aluno(alunos, nome)
        if aluno is None:
            print("Aluno não encontrado.")
        else:
            mostrar_aluno(aluno)
    elif opcao == "4":
        estatisticas(alunos)
    elif opcao == "5":
        exportar_relatorio(alunos)
    elif opcao == "0":
        print("Até logo!")
        break
    else:
        print("Opção inválida!")
```

Execução de teste

Entradas de teste – 1ª execução: cadastros, duplicidade, nota inválida, listagem, busca, estatísticas e exportação:

```
=== BOLETIM ESCOLAR DIGITAL ===
1 - Cadastrar aluno
2 - Listar boletim
3 - Buscar aluno
4 - Estatísticas da turma
5 - Exportar relatório
0 - Sair
Escolha: 1
Nome do aluno: ana lima
Nota 1 (0 a 10): 8
Nota 2 (0 a 10): 7
Nota 3 (0 a 10): 9
Aluno cadastrado e salvo!

=== BOLETIM ESCOLAR DIGITAL ===
1 - Cadastrar aluno
2 - Listar boletim
3 - Buscar aluno
4 - Estatísticas da turma
5 - Exportar relatório
```

```
0 - Sair
Escolha: 1
Nome do aluno: bruno souza
Nota 1 (0 a 10): 4
Nota 2 (0 a 10): 11
Nota inválida! Digite um valor entre 0 e 10.
Nota 2 (0 a 10): 5
Nota 3 (0 a 10): 3.5
Aluno cadastrado e salvo!

=== BOLETIM ESCOLAR DIGITAL ===
1 - Cadastrar aluno
2 - Listar boletim
3 - Buscar aluno
4 - Estatísticas da turma
5 - Exportar relatório
0 - Sair
Escolha: 1
Nome do aluno: carla dias
Nota 1 (0 a 10): 6
Nota 2 (0 a 10): 5
Nota 3 (0 a 10): 7
Aluno cadastrado e salvo!

=== BOLETIM ESCOLAR DIGITAL ===
1 - Cadastrar aluno
2 - Listar boletim
3 - Buscar aluno
4 - Estatísticas da turma
5 - Exportar relatório
0 - Sair
Escolha: 1
Nome do aluno: ANA LIMA
Aluno já cadastrado!

=== BOLETIM ESCOLAR DIGITAL ===
1 - Cadastrar aluno
2 - Listar boletim
3 - Buscar aluno
4 - Estatísticas da turma
5 - Exportar relatório
0 - Sair
Escolha: 2

--- BOLETIM ---
Ana Lima | Notas: 8.0 / 7.0 / 9.0 | Média: 8.00 | Aprovado
Bruno Souza | Notas: 4.0 / 5.0 / 3.5 | Média: 4.17 | Reprovado
Carla Dias | Notas: 6.0 / 5.0 / 7.0 | Média: 6.00 | Recuperação

=== BOLETIM ESCOLAR DIGITAL ===
1 - Cadastrar aluno
2 - Listar boletim
3 - Buscar aluno
4 - Estatísticas da turma
```

```
5 - Exportar relatório
0 - Sair
Escolha: 3
Nome: ana lima
Ana Lima | Notas: 8.0 / 7.0 / 9.0 | Média: 8.00 | Aprovado

=== BOLETIM ESCOLAR DIGITAL ===
1 - Cadastrar aluno
2 - Listar boletim
3 - Buscar aluno
4 - Estatísticas da turma
5 - Exportar relatório
0 - Sair
Escolha: 3
Nome: xyz
Aluno não encontrado.

=== BOLETIM ESCOLAR DIGITAL ===
1 - Cadastrar aluno
2 - Listar boletim
3 - Buscar aluno
4 - Estatísticas da turma
5 - Exportar relatório
0 - Sair
Escolha: 4

--- ESTATÍSTICAS DA TURMA ---
Média da turma: 6.06
Maior média: Ana Lima (8.00)
Aprovado: 1
Recuperação: 1
Reprovado: 1

=== BOLETIM ESCOLAR DIGITAL ===
1 - Cadastrar aluno
2 - Listar boletim
3 - Buscar aluno
4 - Estatísticas da turma
5 - Exportar relatório
0 - Sair
Escolha: 5
Relatório gravado em relatorio.txt.

=== BOLETIM ESCOLAR DIGITAL ===
1 - Cadastrar aluno
2 - Listar boletim
3 - Buscar aluno
4 - Estatísticas da turma
5 - Exportar relatório
0 - Sair
Escolha: 9
Opção inválida!

=== BOLETIM ESCOLAR DIGITAL ===
```

```
1 - Cadastrar aluno
2 - Listar boletim
3 - Buscar aluno
4 - Estatísticas da turma
5 - Exportar relatório
0 - Sair
Escolha: 0
Até logo!
```

Entradas de teste – 2ª execução: os alunos foram carregados do arquivo (persistência):

```
=== BOLETIM ESCOLAR DIGITAL ===
1 - Cadastrar aluno
2 - Listar boletim
3 - Buscar aluno
4 - Estatísticas da turma
5 - Exportar relatório
0 - Sair
Escolha: 2

--- BOLETIM ---
Ana Lima | Notas: 8.0 / 7.0 / 9.0 | Média: 8.00 | Aprovado
Bruno Souza | Notas: 4.0 / 5.0 / 3.5 | Média: 4.17 | Reprovado
Carla Dias | Notas: 6.0 / 5.0 / 7.0 | Média: 6.00 | Recuperação

=== BOLETIM ESCOLAR DIGITAL ===
1 - Cadastrar aluno
2 - Listar boletim
3 - Buscar aluno
4 - Estatísticas da turma
5 - Exportar relatório
0 - Sair
Escolha: 0
Até logo!
```

Conteúdo do arquivo `relatorio.txt` gerado:

```
RELATÓRIO DA TURMA
Ana Lima;8.00;Aprovado
Bruno Souza;4.17;Reprovado
Carla Dias;6.00;Recuperação
```

Casos de teste para correção

Entrada / situação	Resultado esperado
Cadastrar Ana Lima (8, 7, 9)	Média 8,00 – Aprovado
Cadastrar Bruno Souza (4, “11”, 5, 3,5)	“Nota inválida!” para 11; média 4,17 – Reprovado
Cadastrar Carla Dias (6, 5, 7)	Média 6,00 – Recuperação
Cadastrar “ANA LIMA” novamente	“Aluno já cadastrado!” (nome padronizado)
Buscar “ana lima” e “xyz”	Mostra o boletim de Ana; “Aluno não encontrado.”
Estatísticas	Média da turma 6,06; maior média Ana Lima (8,00); 1 Aprovado, 1 Recuperação, 1 Reprovado
Exportar relatório	`relatorio.txt` com 3 linhas `nome;média;situação`
Fechar e reabrir o programa	Os 3 alunos continuam na listagem
Listar com `boletim.txt` inexistente	“Nenhum aluno cadastrado.” (sem erro)

Matriz de correção

Critério	Atende plenamente	Atende parcialmente	Não atende
Funcionamento	Todas as opções funcionam e conferem com os casos de teste.	Uma opção com falha (ex.: busca ou estatísticas).	Menu não executa as operações.
Persistência	Carrega e salva corretamente; relatório gerado; tratamento do arquivo inexistente.	Salva, mas não carrega (ou o contrário), ou relatório incompleto.	Dados perdidos ao fechar.
Funções e estruturas	Funções coesas; lista de dicionários e notas em lista.	Funções com responsabilidades misturadas ou estruturas pouco adequadas.	Sem funções ou sem estruturas de dados.
Regras de negócio	RN01 a RN06 implementadas.	Faltam 1 ou 2 regras (ex.: nome repetido ou `;`).	Média ou situação incorretas.
Organização e colaboração	Legível, comentado, testes e registro da dupla completos.	Falta registro ou testes.	Sem organização ou colaboração.

Erros frequentes e orientações

- **Abrir o arquivo em modo `"w"` ao cadastrar sem regravar a lista inteira**, apagando os alunos anteriores.

- **Esquecer a conversão `float()`** ao ler as notas do arquivo (a média vira concatenação de textos ou gera TypeError).
- **Esquecer o `.strip()`** ao ler as linhas: o `\n` fica grudado na última nota.
- **Executar o programa em uma pasta diferente** da do arquivo (VS Code sem abrir a pasta; Colab sem montar o Drive).
- **Não tratar o arquivo inexistente** na primeira execução (FileNotFoundError).
- **Dividir a média por 3 fixo em vez de `len(notas)`** — funciona, mas dificulta alterações.

Orientações para os desafios extras

- **Ranking:** `ranking = sorted(alunos, key=calcular_media, reverse=True)` e percorrer a lista resultante com `for`. Aceite também ordenação manual (por trocas).
- **Média ponderada:** `(n[0] * 2 + n[1] * 3 + n[2] * 5) / 10` dentro de `calcular_media`.
- **Edição/remoção:** buscar o aluno com `buscar_aluno`; para remover, `alunos.remove(aluno)` e chamar `salvar_alunos(alunos)`.

Gabarito – Projeto 5: Amazônia Bank: Mini-Banco Digital

Solução de referência

```
# Projeto 5 - Amazônia Bank: Mini-Banco Digital (valores fictícios)
import os

ARQUIVO = "contas.txt"

class Conta:
    def __init__(self, titular, senha, saldo=0.0):
        self.titular = titular
        self.senha = senha
        self.saldo = saldo
        self.historico = []

    def autenticar(self, senha):
        return self.senha == senha

    def depositar(self, valor):
        if valor <= 0:
            return False
        self.saldo += valor
        self.historico.append(f"Depósito: + R$ {valor:.2f}")
        return True

    def sacar(self, valor):
        if valor <= 0 or valor > self.saldo:
            return False
        self.saldo -= valor
        self.historico.append(f"Saque: - R$ {valor:.2f}")
        return True

    def transferir(self, destino, valor):
        if valor <= 0 or valor > self.saldo:
            return False
        self.saldo -= valor
        destino.saldo += valor
        self.historico.append(f"Transferência para {destino.titular}: - R$
{valor:.2f}")
        destino.historico.append(f"Transferência de {self.titular}: + R$
{valor:.2f}")
        return True

    def extrato(self):
        print(f"\n--- EXTRATO DE {self.titular} ---")
        if len(self.historico) == 0:
            print("Sem movimentações nesta sessão.")
        for movimento in self.historico:
            print(movimento)
        print(f"Saldo atual: R$ {self.saldo:.2f}")
```

```
def carregar_contas():
    contas = {}
    if os.path.exists(ARQUIVO):
        with open(ARQUIVO, "r", encoding="utf-8") as f:
            for linha in f:
                titular, senha, saldo = linha.strip().split(";")
                contas[titular] = Conta(titular, senha, float(saldo))
    return contas

def salvar_contas(contas):
    with open(ARQUIVO, "w", encoding="utf-8") as f:
        for conta in contas.values():
            f.write(f"{conta.titular};{conta.senha};{conta.saldo:.2f}\n")

def criar_conta(contas):
    titular = input("Nome do titular: ").strip().title()
    if titular == "" or ";" in titular:
        print("Nome inválido.")
        return
    if titular in contas:
        print("Já existe uma conta para este titular.")
        return
    senha = input("Senha (4 dígitos): ")
    if len(senha) != 4 or not senha.isdigit():
        print("A senha deve ter exatamente 4 dígitos numéricos.")
        return
    contas[titular] = Conta(titular, senha)
    salvar_contas(contas)
    print(f"Conta criada com sucesso para {titular}!")

def menu_conta(conta, contas):
    while True:
        print(f"\n--- Conta de {conta.titular} ---")
        print("1 - Saldo")
        print("2 - Depositar")
        print("3 - Sacar")
        print("4 - Transferir")
        print("5 - Extrato")
        print("0 - Sair da conta")
        opcao = input("Escolha: ")

        if opcao == "1":
            print(f"Saldo: R$ {conta.saldo:.2f}")
        elif opcao == "2":
            valor = float(input("Valor do depósito (R$): "))
            if conta.depositar(valor):
                salvar_contas(contas)
                print("Depósito realizado!")
            else:
                print("Valor inválido.")
        elif opcao == "3":
            valor = float(input("Valor do saque (R$): "))
            if conta.sacar(valor):
```

```
                salvar_contas(contas)
                print("Saque realizado!")
            else:
                print("Saque não realizado: valor inválido ou saldo
insuficiente.")
        elif opcao == "4":
            nome_destino = input("Titular de destino: ").strip().title()
            if nome_destino not in contas:
                print("Conta de destino não encontrada.")
                continue
            if nome_destino == conta.titular:
                print("Não é possível transferir para a própria conta.")
                continue
            valor = float(input("Valor da transferência (R$): "))
            if conta.transferir(contas[nome_destino], valor):
                salvar_contas(contas)
                print("Transferência realizada!")
            else:
                print("Transferência não realizada: valor inválido ou saldo
insuficiente.")
        elif opcao == "5":
            conta.extrato()
        elif opcao == "0":
            break
        else:
            print("Opção inválida.")

def principal():
    contas = carregar_contas()
    while True:
        print("\n=== AMAZÔNIA BANK - MINI-BANCO DIGITAL ===")
        print("1 - Criar conta")
        print("2 - Entrar")
        print("0 - Sair")
        opcao = input("Escolha: ")

        if opcao == "1":
            criar_conta(contas)
        elif opcao == "2":
            titular = input("Titular: ").strip().title()
            senha = input("Senha: ")
            if titular in contas and contas[titular].autenticar(senha):
                print(f"Bem-vindo(a), {titular}!")
                menu_conta(contas[titular], contas)
            else:
                print("Titular ou senha inválidos.")
        elif opcao == "0":
            print("Até logo!")
            break
        else:
            print("Opção inválida.")

principal()
```

Execução de teste

Entradas de teste – 1ª execução: criação de contas, validações, login, operações, extrato:

```
=== AMAZÔNIA BANK - MINI-BANCO DIGITAL ===
1 - Criar conta
2 - Entrar
0 - Sair
Escolha: 1
Nome do titular: marina silva
Senha (4 dígitos): 1234
Conta criada com sucesso para Marina Silva!

=== AMAZÔNIA BANK - MINI-BANCO DIGITAL ===
1 - Criar conta
2 - Entrar
0 - Sair
Escolha: 1
Nome do titular: joão pedro
Senha (4 dígitos): 12ab
A senha deve ter exatamente 4 dígitos numéricos.

=== AMAZÔNIA BANK - MINI-BANCO DIGITAL ===
1 - Criar conta
2 - Entrar
0 - Sair
Escolha: 1
Nome do titular: JOÃO PEDRO
Senha (4 dígitos): 4321
Conta criada com sucesso para João Pedro!

=== AMAZÔNIA BANK - MINI-BANCO DIGITAL ===
1 - Criar conta
2 - Entrar
0 - Sair
Escolha: 1
Nome do titular: marina silva
Já existe uma conta para este titular.

=== AMAZÔNIA BANK - MINI-BANCO DIGITAL ===
1 - Criar conta
2 - Entrar
0 - Sair
Escolha: 2
Titular: marina silva
Senha: 0000
Titular ou senha inválidos.

=== AMAZÔNIA BANK - MINI-BANCO DIGITAL ===
1 - Criar conta
2 - Entrar
0 - Sair
Escolha: 2
Titular: marina silva
Senha: 1234
```

```
Bem-vindo(a), Marina Silva!

--- Conta de Marina Silva ---
1 - Saldo
2 - Depositar
3 - Sacar
4 - Transferir
5 - Extrato
0 - Sair da conta
Escolha: 2
Valor do depósito (R$): 500
Depósito realizado!

--- Conta de Marina Silva ---
1 - Saldo
2 - Depositar
3 - Sacar
4 - Transferir
5 - Extrato
0 - Sair da conta
Escolha: 3
Valor do saque (R$): 800
Saque não realizado: valor inválido ou saldo insuficiente.

--- Conta de Marina Silva ---
1 - Saldo
2 - Depositar
3 - Sacar
4 - Transferir
5 - Extrato
0 - Sair da conta
Escolha: 3
Valor do saque (R$): 120.50
Saque realizado!

--- Conta de Marina Silva ---
1 - Saldo
2 - Depositar
3 - Sacar
4 - Transferir
5 - Extrato
0 - Sair da conta
Escolha: 2
Valor do depósito (R$): -5
Valor inválido.

--- Conta de Marina Silva ---
1 - Saldo
2 - Depositar
3 - Sacar
4 - Transferir
5 - Extrato
0 - Sair da conta
Escolha: 4
```

```
Titular de destino: xingu
Conta de destino não encontrada.

--- Conta de Marina Silva ---
1 - Saldo
2 - Depositar
3 - Sacar
4 - Transferir
5 - Extrato
0 - Sair da conta
Escolha: 4
Titular de destino: marina silva
Não é possível transferir para a própria conta.

--- Conta de Marina Silva ---
1 - Saldo
2 - Depositar
3 - Sacar
4 - Transferir
5 - Extrato
0 - Sair da conta
Escolha: 4
Titular de destino: joão pedro
Valor da transferência (R$): 200
Transferência realizada!

--- Conta de Marina Silva ---
1 - Saldo
2 - Depositar
3 - Sacar
4 - Transferir
5 - Extrato
0 - Sair da conta
Escolha: 1
Saldo: R$ 179.50

--- Conta de Marina Silva ---
1 - Saldo
2 - Depositar
3 - Sacar
4 - Transferir
5 - Extrato
0 - Sair da conta
Escolha: 5

--- EXTRATO DE Marina Silva ---
Depósito: + R$ 500.00
Saque: - R$ 120.50
Transferência para João Pedro: - R$ 200.00
Saldo atual: R$ 179.50

--- Conta de Marina Silva ---
1 - Saldo
2 - Depositar
```

```
3 - Sacar
4 - Transferir
5 - Extrato
0 - Sair da conta
Escolha: 0

=== AMAZÔNIA BANK - MINI-BANCO DIGITAL ===
1 - Criar conta
2 - Entrar
0 - Sair
Escolha: 0
Até logo!
```

Entradas de teste – 2ª execução: saldo de João Pedro foi carregado do arquivo (o extrato da sessão anterior não persiste):

```
=== AMAZÔNIA BANK - MINI-BANCO DIGITAL ===
1 - Criar conta
2 - Entrar
0 - Sair
Escolha: 2
Titular: joão pedro
Senha: 4321
Bem-vindo(a), João Pedro!

--- Conta de João Pedro ---
1 - Saldo
2 - Depositar
3 - Sacar
4 - Transferir
5 - Extrato
0 - Sair da conta
Escolha: 1
Saldo: R$ 200.00

--- Conta de João Pedro ---
1 - Saldo
2 - Depositar
3 - Sacar
4 - Transferir
5 - Extrato
0 - Sair da conta
Escolha: 5

--- EXTRATO DE João Pedro ---
Sem movimentações nesta sessão.
Saldo atual: R$ 200.00

--- Conta de João Pedro ---
1 - Saldo
2 - Depositar
3 - Sacar
4 - Transferir
5 - Extrato
0 - Sair da conta
```

```
Escolha: 0

=== AMAZÔNIA BANK - MINI-BANCO DIGITAL ===
1 - Criar conta
2 - Entrar
0 - Sair
Escolha: 0
Até logo!
```

Conteúdo do arquivo `contas.txt` ao final:

```
Marina Silva;1234;179.50
João Pedro;4321;200.00
```

Casos de teste para correção

Entrada / situação	Resultado esperado
Criar “marina silva”, senha 1234	Conta criada para “Marina Silva”
Criar “joão pedro”, senha “12ab”	“A senha deve ter exatamente 4 dígitos numéricos.”
Criar “JOÃO PEDRO”, senha 4321	Conta criada para “João Pedro”
Criar “marina silva” novamente	“Já existe uma conta para este titular.”
Login com senha 0000	“Titular ou senha inválidos.”
Depositar 500; sacar 800; sacar 120,50; depositar −5	Depósito ok; saque recusado (saldo insuficiente); saque ok; valor inválido
Transferir para “xingu”; para si mesma; R$ 200 para João Pedro	Destino não encontrado; recusada; transferência realizada
Saldo e extrato de Marina	Saldo R$ 179,50; extrato com depósito, saque e transferência
Reiniciar e entrar como João Pedro	Saldo R$ 200,00 (persistido); extrato vazio nesta sessão
`contas.txt` ao final	Marina Silva;1234;179.50 e João Pedro;4321;200.00

Matriz de correção

Critério	Atende plenamente	Atende parcialmente	Não atende
Funcionamento	Criação, login e todas as operações corretas nos casos de teste.	Uma operação com falha (ex.: transferência).	Não conclui o fluxo de login ou das operações.
POO	Classe `Conta` com atributos e métodos coerentes; métodos devolvem `True`/`False`.	Usa classe, mas com lógica fora dos métodos ou atributos mal definidos.	Sem classe, ou classe sem uso real.
Persistência	Contas e saldos salvos e recarregados corretamente.	Salva, mas não carrega (ou o saldo se perde em alguma operação).	Dados perdidos ao fechar.
Regras de negócio	RN01 a RN07 implementadas.	Faltam 1 ou 2 regras (ex.: transferência para si mesma).	Permite saldo negativo ou conta duplicada.
Organização, testes e apresentação	Legível, quadro com 8+ casos, apresentação clara.	Poucos testes ou explicação incompleta.	Sem testes ou sem domínio do próprio código.

Erros frequentes e orientações

- **Esquecer o `self`** nos métodos ou escrever `saldo += valor` em vez de `self.saldo += valor`.
- **Salvar o arquivo só ao sair**: se o programa for interrompido, as operações são perdidas.
- **Comparar a senha com `==` sem tratar espaços** ou aceitar senhas de tamanho diferente.
- **Transferir sem atualizar a conta de destino** ou sem passar o **objeto** (e não o nome) como `destino`.
- **Gravar o saldo com `float` sem formatação**, gerando valores como `179.49999999999997`: use `:.2f`.
- **Não verificar `nome_destino in contas`**, causando `KeyError`.
- Mostrar a mensagem “realizado” dentro do método e de novo no menu (duplicidade): o método deve apenas devolver `True`/`False`.

Orientações para os desafios extras

- **Histórico em arquivo:** gravar um arquivo `movimentos.txt` com `titular;descricao` (modo `"a"`) e carregá-lo para o `historico` de cada conta ao iniciar.

- **Tentativas:** contador `tentativas` no login; `if tentativas == 3: print("Acesso bloqueado.")` e encerrar o fluxo.
- **Tarifa:** em `sacar`, `total = valor + 2.0` e comparar `total > self.saldo`; debitar `total`.
- **Classe Banco:** atributo `contas`; métodos `carregar()`, `salvar()`, `criar_conta()` e `login()` no lugar das funções soltas.